

CS3334 Final Report

Name: STEFANOVIC Zlata
SID: 58407472 | EID: zstefanov2

December 5, 2024

All of the programs can be compiled with `-std=c++11`. Optionally, you may use `"g++ -std=c++11 -O2 897hashing.cpp -o hashing"` to add optimization. For this assignment, I used the following methods:

1. **Brute force:** We go through each element in the array and count its frequency. Then, compare all the frequencies and find the most common element. Going through each element and counting its frequency has $O(n^2)$ complexity, both in the worst and best case scenario. Going through the arrays of their frequencies and finding the most common one takes $O(n)$ time. Therefore, the total time complexity for T test examples is $O(Tn^2)$. It works the best in the case where all the numbers are the same, for example "1 1 1 1 1". The time complexity will still be $O(n^2)$ per test case, however, the inner loop for finding the most frequent element will have slightly less operations. In the worst case all the elements would be different so we would go through each operation in the inner loop. However, the complexity will still be $O(n^2)$ per test case.
2. **Hashing:** We make a hash map for the elements, where the key is the element and the value is the count of the element. I used an unordered map for this. As we read each integer, we increase the value of the hash map. Then we go through the array of integers again and compare their hash values to find the most frequent element. The insertion and the updating of the hash map both take $O(1)$ time on average, so doing this for all integers would take $O(n)$. After that, going through the array again to compare the hash values takes $O(n)$ times. This means that the overall complexity is $O(n)$, therefore, if we have T examples it would take us $O(Tn)$ time in the average/best case scenario. Opposite to brute force, the program will perform worst if all elements in the input are the same, or map to the same hash, depending on the implementation of the hashing algorithm. For `unordered_map` in C++ it happens when the input data is big and input values are multiples of 107897 or 126271 (depending on the compiler) a lot of collisions take place. This increases the time complexity for insertion and lookup in the hash map, the worst time complexity in this case would be $O(n^2)$ per test example. So in the worst case, the complexity will be $O(Tn^2)$.
3. **AVL tree:** For each integer we read from the input, we check if it exists in the AVL tree and update its count, maintaining balance. After counting, we go through the tree to determine which integer appeared the most times. The operations of the AVL

tree (insertion and retrieval) each take $O(\log n)$ time, making the overall complexity of processing an input line $O(n \log n)$, where n is the number of integers on that line. If we have T test examples, the complexity of the whole program would be $O(Tn \log n)$. Even in the worst case, where the input is the same repeating digit such as "1 1 1 1 1 ...", the AVL tree will keep a logarithmic height and thus keep the time complexity the same.

In conclusion, on average the hashing method outperforms brute force and AVL tree. However, in a specific case where we have input data that is big and input values are multiples of 107897 or 126271, if we have a lot of collisions the time complexity of the hash method will jump to $O(n^2)$ whereas the AVL tree complexity will stay $O(n \log n)$ outperforming the hash. Lastly, if the simplicity of our code is more important than time complexity, we might decide to use the brute force method.